

2d_exercises

January 22, 2025

1 Digital Biomarkers Project

The goal of this project is to develop a pipeline for analyzing breast ultrasound images, extracting meaningful features, and classifying tumors into benign or malignant categories. By leveraging medical imaging, feature extraction, and machine learning, the project demonstrates the practical applications of digital biomarkers in personalized healthcare.

Tasks:

- Data Preparation: Organizing ultrasound images and masks for analysis.
- Feature Extraction: Deriving numerical descriptors of tumor properties.
- Classification: Using machine learning to predict tumor type based on extracted features.

2 Data Preparation

The first step is to prepare a clean, structured dataset of benign and malignant tumor images and segmentation masks.

For this, I download the dataset, try to understand its structure, and preprocessing it for analysis. Using the csv provided, I read the image into four lists: benignant, malignant and the corresponding segmentation mask, which identifies the tumor's location within the ultrasound image. The images are visualized to ensure correctness and alignment between the ultrasound image and its mask.

2.1 Imports

```
[3]: import os
import re
import sys
import contextlib
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import random
from PIL import Image
from typing import List, Tuple
from skimage.feature import graycomatrix, graycoprops
from tpot import TPOTClassifier
```

```

from sklearn.metrics import classification_report, confusion_matrix, \
    roc_auc_score, roc_curve, auc, accuracy_score, precision_score, \
    recall_score, f1_score, precision_recall_curve
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier

```

2.2 Loading the images

I will perform the following steps to load the images:

1. Paths: Define the paths to the benign, malignant, and normal folders
2. Initialize empty lists to store images and labels
3. Define a function to load the images and labels

2.2.1 Paths

In the first step, I save the paths to the folders containing the different datasets to variables so I can use them later on.

```

[4]: base_dir = "Breast tumor/Dataset_BUSI_with_GT"
    benign_dir = os.path.join(base_dir, "benign")
    malignant_dir = os.path.join(base_dir, "malignant")
    normal_dir = os.path.join(base_dir, "normal")

```

Then, I load the valid indices from the csv file. This file contains the indices of images and masks that are considered valid. Some are not, this seems to be due to text / other distractors that could interfere with the feature extraction.

```

[5]: valid_indices_df = pd.read_csv('Breast tumor/Benign_malignant_indices.csv')

    benign_indices = valid_indices_df['benign'].dropna().astype(int).tolist()
    malignant_indices = valid_indices_df['malignant'].dropna().astype(int).tolist()

```

During initial analysis of the images, I realised we have images that contain several breast tumours in one image. This was visible because I tried to import all masks and images and I had one more mask than images (of the benign images kind). It was image 181 and I ended up taking it out of the project.

2.2.2 Define a function to load the images and labels

With this function, I can load the images into the lists. We pass the category benign / malignant as an argument so we can use it to load both. The function returns two lists: the images and the corresponding masks.

Problems / Goals:

- separate images and masks

- make sure only valid images are loaded (see the csv)
- later edit: make sure we have the same amount of masks and images. & only include images we want to keep -> in the valid list of benign images, there was the image 181 included, which contained several tumours/several masks. I assumed this was something we wanted to exclude and removed the image 181 and it's mask.

```
[6]: exclude_benign = [181]
benign_indices = [index for index in benign_indices if index not in
↳exclude_benign]

[7]: def read_images(folder_path, category, valid_indices) -> Tuple[List[str],
↳List[str]]:
    images = []
    masks = []

    for index in valid_indices:
        # Construct the filenames based on the index and category
        img_file_name = f"{category} ({index}).png"
        mask_file_name = f"{category} ({index})_mask.png"

        # Define the full file paths
        img_file_path = os.path.join(folder_path, img_file_name)
        mask_file_path = os.path.join(folder_path, mask_file_name)
        # Check if the files exist
        if not os.path.exists(img_file_path) or not os.path.
↳exists(mask_file_path):
            print(img_file_path)
            print(mask_file_path)
            raise FileNotFoundError(f"Image or mask file missing for index
↳{index}")

        # Read the image and mask, and append them to the lists
        images.append(plt.imread(img_file_path)[: , : , 0])
        masks.append(plt.imread(mask_file_path))

    return images, masks
```

Here we call the function for both benign and malignant images.

```
[8]: benign_images, benign_masks = read_images(benign_dir, 'benign', benign_indices)
malignant_images, malignant_masks = read_images(malignant_dir, 'malignant',
↳malignant_indices)
```

2.2.3 Check whether Images are correctly loaded

To verify the previous steps, I display the first item in each list.

```
[13]: import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(5, 5)) # Create a 2x2 grid

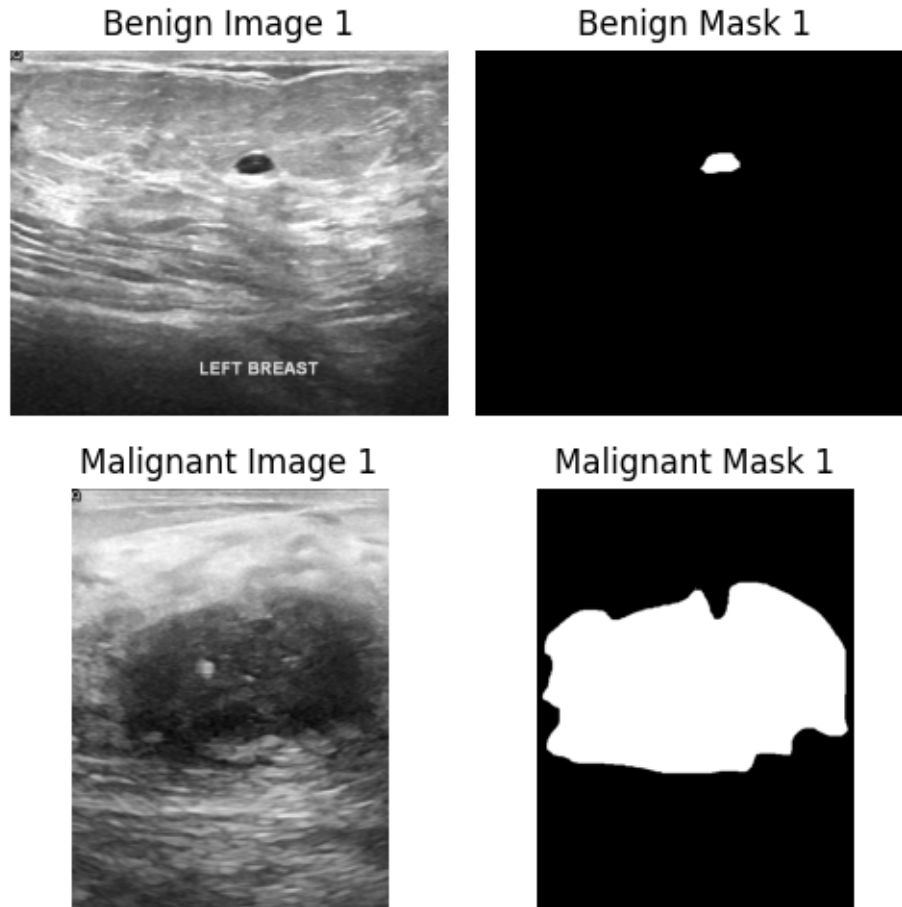
# Benign image and mask
axes[0, 0].imshow(benign_images[0], cmap="gray")
axes[0, 0].set_title("Benign Image 1")
axes[0, 0].axis("off")

axes[0, 1].imshow(benign_masks[0], cmap="gray")
axes[0, 1].set_title("Benign Mask 1")
axes[0, 1].axis("off")

# Malignant image and mask
axes[1, 0].imshow(malignant_images[0], cmap="gray")
axes[1, 0].set_title("Malignant Image 1")
axes[1, 0].axis("off")

axes[1, 1].imshow(malignant_masks[0], cmap="gray")
axes[1, 1].set_title("Malignant Mask 1")
axes[1, 1].axis("off")

plt.tight_layout() # Adjust spacing to prevent overlap
plt.show()
```



I also want to make sure I only loaded valid images. To do this, I compare the length of the valid indices benign / malignant with the length of the image lists.

```
[15]: print(f"Valid Benign Indices: {len(valid_indices_df['benign'])}")
      print(f"Benign Images: {len(benign_images)}")
      print(f"Benign Masks: {len(benign_masks)}")
      print(f"Valid Malignant Indices: {len(valid_indices_df['malignant'])}")
      print(f"Malignant Images: {len(malignant_images)}")
      print(f"Malignant Masks: {len(malignant_masks)}")
```

```
Valid Benign Indices: 176
Benign Images: 175
Benign Masks: 175
Valid Malignant Indices: 176
Malignant Images: 176
Malignant Masks: 176
```

Here we can see that I now have one less image for the benign images (as I removed the one with the two tumours / masks). If necessary, I will later drop out one of the malignant images at random.

3 Image Preprocessing

One could perform many methods of image preprocessing such as: - Geometric Preprocessing - Resizing, Cropping, Rotating, Flipping: adjust image dimensions and orientation - Contrast & Intensity Adjustments - Grayscale: from colored to grayscale - Windowing: change intensity ranges - Filtering & Noise Reduction - Gaussian & Average Filters: smooth noise - Median & Bilateral Filtering: reduces noise while preserving edges - Edge Detection (e.g. Sobel): edge detection - Thresholding & Binarization: e.g. convert grayscale to binary

I just wanted to see how the model would perform without doing too much preprocessing so I just left it for an initial model. As the performance was quite acceptable.

Later on, I tried to do Normalization, Histogram Equalization and applied a Gaussian Filter to reduce noise but the performance was not significantly better -> SVM got better, while RF got worse. In the end, I decided to leave this and do not perform any Image Preprocessing (as our dataset is already very nice, e.g. we have the masks for the tumors etc.).

```
[ ]: from skimage.feature import graycomatrix
      from skimage.color import rgb2gray
      from skimage.exposure import equalize_hist
      from skimage.filters import gaussian

      def preprocess_image(image):
          if len(image.shape) == 3:
              image = rgb2gray(image) # Convert to grayscale (to make sure, but
              ↪should already be grayscale)

              # Normalize to [0, 1]
              image = image / image.max() if image.max() > 1 else image

              # Apply histogram equalization
              image = equalize_hist(image)

              # Apply Gaussian filter to reduce noise
              image = gaussian(image, sigma=1)

          return (image * 255).astype(np.uint8) # Convert back to uint8
```

4 Feature Extraction

In this part, I try to extract science-based features from the images / masks.

4.1 Shape Features

I try to extract the following features from the images:

- Area (Volume): number of pixels within tumor
- Perimeter: this expresses the total surface of the tumor
- Circularity: round vs. irregular shaped?

- Regularity

To calculate these geometric features, we can use the masks (and don't need the images themselves.)

Edit: I also tried other features, but these were the most successful.

4.1.1 Area

The area describes the number of pixels contained in the tumor / mask. So we can return the sum of all values in the mask that are > 0 .

```
[9]: def calculate_area(mask):
      return np.sum(mask > 0)
```

4.1.2 Perimeter

Here, I calculate the boundary of the mask to get the surface of the tumor.

```
[10]: def calculate_perimeter(mask):
        #to boolean array -> after this, 1 is object and 0 is background
        mask = mask > 0
        # Compute the boundary (edges) of the mask; first line is vertical edges,
        ↪second line is horizontal edges
        boundary = np.logical_xor(mask, np.pad(mask[:-1, :], ((1, 0), (0, 0)),
        ↪mode='constant')) | \
                    np.logical_xor(mask, np.pad(mask[:, :-1], ((0, 0), (1, 0)),
        ↪mode='constant'))
        return np.sum(boundary)
```

4.1.3 Circularity

Here we calculate the circularity of the tumor by comparing the tumor's area and perimeter. We compare the shape of our tumor to the perfect circle, because the perfect circle would have the smallest perimeter for a given area. There is a formula to express this: $\text{Circularity} = (4 \cdot \pi \cdot \text{area}) / (\text{perimeter}^2)$

The circularity of a perfect circle would be 1. The closer to one the circularity of our tumour is, the closer to a perfect circle (= the more round) it is.

```
[11]: def calculate_circularity(area, perimeter):
        if perimeter == 0: # this is to avoid dividing by zero
            return 0
        return (4 * np.pi * area) / (perimeter ** 2)
```

4.1.4 Regularity

Here, we calculate how irregular the "border" of our tumour is. We do this by comparing the area of the tumor to the area of the smallest rectangle that fully surrounds the tumor (= bounding box). This also results in a value between 0 (very irregular) and 1 (very rectangular, fills the bounding box).

```
[12]: def calculate_regularity(mask):
    mask = mask > 0 # to boolean array -> 0/False is background, 1/True is
    ↪object
    rows = np.any(mask, axis=1) # returns True for rows that contain at least
    ↪one "True" value = one part of tumour => sum of all these rows= height of
    ↪tumour
    cols = np.any(mask, axis=0) # see above but for width
    bounding_box_area = (rows.sum() * cols.sum()) # height * width = area of
    ↪smallest rectangle that can contain the tumour
    if bounding_box_area == 0: # to avoid dividing by zero (would be an empty
    ↪mask)
        return 0
    return np.sum(mask) / bounding_box_area
```

4.1.5 Shape Feature Extraction

Next, I define a function that runs the 4 previously defined functions and creates a df with the results.

```
[13]: def extract_shape_features(masks):
    #initialize empty lists
    areas = []
    perimeters = []
    circularities = []
    regularities = []

    for mask in masks: #iterates over all masks
        area = calculate_area(mask)
        perimeter = calculate_perimeter(mask)
        circularity = calculate_circularity(area, perimeter)
        regularity = calculate_regularity(mask)

        # append the calculated values to the lists
        areas.append(area)
        perimeters.append(perimeter)
        circularities.append(circularity)
        regularities.append(regularity)

    # create df from lists
    df = pd.DataFrame({
        'area': areas,
        'perimeter': perimeters,
        'circularity': circularities,
        'regularity': regularities
    })

    return df
```

Then I run this code with both lists, the benign masks and malignant masks and save the result as dfs.

```
[14]: benign_df = extract_shape_features(benign_masks)
      malignant_df = extract_shape_features(malignant_masks)
```

4.2 Texture Features

I will extract the following features:

1st order Texture Features: -> from the image directly

- Mean Intensity: Average brightness
- Standard Deviation: Intensity variability or spread

2nd order Texture Features: -> from the Gray-Level Co-Occurrence Matrix (GLCM)

- Contrast
- Entropy
- Homogeneity

I am using the graycomatrix function from the skimage (Scikit-Image) library to compute the GLCM.

4.2.1 1st Statistical Order

Mean Intensity Computes the average intensity of the entire image. This indicates a baseline “brightness” or intensity of the image, with higher values indicating a brighter image.

```
[15]: def calculate_mean_intensity(image):
      return np.mean(image)
```

Standard Deviation Measures the standard deviation of the grayscale values in the image. This gives us a feeling of the spread / variability of intensity values / gray scales contained in the image.

```
[16]: def calculate_standard_deviation(image):
      return np.std(image)
```

4.2.2 2nd Statistical Order

These features are computed using the Gray level co-occurrence matrix (GLCM). I use the graycomatrix function from the Scikit-Image library to compute it. Then, I compute different features from the matrix.

Contrast Measures the difference in intensity between neighboring pixels as described by the gray-level co-occurrence matrix (GLCM).

Higher values mean that the contrast / transition is higher and sharper, while low values indicate smooth / uniform texture.

```
[17]: def calculate_contrast(glcm):
      return graycoprops(glcm, 'contrast')[0, 0]
```

Homogeneity This measures, how “smooth” or “similar” a texture is. High homogeneity indicates a smooth texture with minimal intensity variation. Low homogeneity suggests irregular or abrupt intensity changes.

```
[18]: def calculate_homogeneity(glcm):  
        return graycoprops(glcm, 'homogeneity')[0, 0]
```

Entropy How random the intensity distribution is in the image. High entropy indicates a diverse and irregular texture. Low entropy suggests uniform or predictable patterns.

```
[19]: def calculate_entropy(glcm):  
        glcm_normalized = glcm / glcm.sum()  
        return -np.sum(glcm_normalized * np.log2(glcm_normalized + np.finfo(float).  
        ↪eps))
```

4.2.3 Texture Feature Extraction

Defining a function that uses the functions to compute the first and second order texture features and adds them as new columns to the datasets.

```
[21]: def extract_texture_features(images, df):  
        contrasts = []  
        homogeneities = []  
        entropies = []  
        mean_intensities = []  
        standard_deviations = []  
  
        for image in images:  
            # Ensure image is grayscale with integer pixel values  
            image = (image * 255).astype(np.uint8) if image.max() <= 1.0 else image  
  
            # Calculate GLCM (Gray-Level Co-occurrence Matrix)  
            glcm = graycomatrix(image, distances=[1], angles=[0], levels=256,  
            ↪symmetric=True, normed=True) #from the skimage.feature library  
  
            # Calculate texture features  
            contrasts.append(calculate_contrast(glcm))  
            homogeneities.append(calculate_homogeneity(glcm))  
            entropies.append(calculate_entropy(glcm))  
  
            # Calculate first-order statistics  
            mean_intensities.append(calculate_mean_intensity(image))  
            standard_deviations.append(calculate_standard_deviation(image))  
  
        # Add texture features to DataFrame  
        df['contrast'] = contrasts  
        df['homogeneity'] = homogeneities  
        df['entropy'] = entropies
```

```
df['mean_intensity'] = mean_intensities
df['standard_deviation'] = standard_deviations

return df
```

```
[22]: benign_df = extract_texture_features(benign_images, benign_df)
malignant_df = extract_texture_features(malignant_images, malignant_df)
```

```
[23]: benign_df.head()
```

```
[23]:
```

	area	perimeter	circularity	regularity	contrast	homogeneity	\
0	984	134	0.688645	0.742081	30.768506	0.360982	
1	7022	344	0.745682	0.749253	26.589997	0.367586	
2	2204	201	0.685535	0.791667	12.313682	0.369451	
3	3997	249	0.810112	0.844853	12.179959	0.381598	
4	32918	711	0.818284	0.799505	13.264486	0.413510	

	entropy	mean_intensity	standard_deviation
0	11.693589	127.688151	72.134465
1	11.621093	127.526840	72.565455
2	11.526682	127.589303	73.100101
3	11.458977	127.623735	73.060041
4	11.323793	127.502854	73.127838

```
[24]: malignant_df.head()
```

```
[24]:
```

	area	perimeter	circularity	regularity	contrast	homogeneity	\
0	90995	1438	0.552980	0.777257	11.596691	0.377117	
1	13990	599	0.489975	0.608525	15.217568	0.353956	
2	11825	560	0.473844	0.572085	15.649819	0.350653	
3	12527	608	0.425843	0.633957	15.719214	0.337915	
4	10196	465	0.592562	0.646626	16.062758	0.343410	

	entropy	mean_intensity	standard_deviation
0	11.493686	127.506819	72.802017
1	11.603921	127.600643	73.137793
2	11.612688	127.590112	73.103042
3	11.692460	127.525821	73.105142
4	11.673063	127.549661	73.071927

I can see that I will need to scale the features in the dataset for some machine learning models, as the magnitude differs quite much between the features.

5 Classification

The goal in this part is to train a classifier that predicts whether an image contains a benign or a malignant tumor.

For this, the following steps need to be taken:

- create a joint df out of `benign_df` and `malignant_df`
- split this df into train/test
- scale features (necessary for logistic regression, SVM, MLP,...)
- train ML models using CV

5.1 Combining Dataframes

Here, I assign the two dataframes their labels that will be the target for our classification task.

```
[25]: benign_df['label'] = 0
      malignant_df['label'] = 1
```

Next, I drop one random instance of the `malignant_df` (because we took out one instance of the `benign_df` due to it containing several tumours and having perfectly balanced classes seems better.)

Then I combine both into one df.

```
[26]: random_index = random.choice(malignant_df.index)
      malignant_df = malignant_df.drop(random_index)

      combined_df = pd.concat([benign_df, malignant_df], ignore_index=True)
```

5.2 Train-Test Split

I decided to perform only a train-test split (rather than train val test split) and use cross-validation for model evaluation due to the limited availability of the data.

```
[27]: X = combined_df.drop('label', axis=1)
      y = combined_df['label']
```

```
[28]: from sklearn.model_selection import train_test_split

      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
      ↪random_state=42, stratify=y)
```

5.2.1 Scaling the Features

Some models that are based on distances (e.g. Logistic Regression, SVM, MLP) need the input features to be scaled, so I will do this and proceed to use the scaled train and test sets for the models that need this.

```
[29]: scaler = StandardScaler()
      X_train_scaled = scaler.fit_transform(X_train)
      X_test_scaled = scaler.transform(X_test)
```

5.3 Training Different ML Models

Now, I try different kinds of machine learning with the dataset I created, I always use 5-fold CV to get the most representative evaluation of the model (and be able to compare the different

models!) with this relatively small dataset, as well as a hyperparameter grid to find the optimal hyperparameters of the model.

5.3.1 Model Performance Logs

To be able to remember / compare the performance, I define two functions: one that logs the performance, and one that will display the model logs.

Logging Function

```
[30]: performance_logs = []

def log_model_performance(model_name, y_true, y_pred):
    global performance_logs

    # Calculate performance metrics
    metrics = {
        'Model': model_name,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, average='weighted'),
        'Recall': recall_score(y_true, y_pred, average='weighted'),
        'F1-Score': f1_score(y_true, y_pred, average='weighted')
    }

    # Append the metrics to the logs
    performance_logs.append(metrics)
```

Display Function

```
[31]: def display_performance_comparison():
    global performance_logs
    performance_df = pd.DataFrame(performance_logs)
    performance_df = performance_df.sort_values(by='F1-Score', ascending=False)

    print("Model Performance Comparison:")
    print(performance_df)

    return performance_df
```

5.3.2 Logistic Regression

First, I tried logistic regression, a relatively simple model. With scaling, the model achieved 92% in CV, which is good.

```
[32]: log_reg = LogisticRegression(max_iter=1000, random_state=42)

# Define the hyperparameter grid
param_grid_log_reg = {
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga']
```

```

}

grid_log_reg = GridSearchCV(estimator=log_reg, param_grid=param_grid_log_reg,
    cv=5, scoring='accuracy')
grid_log_reg.fit(X_train_scaled, y_train)

# Best parameters and best score
print("Best Parameters for Logistic Regression:", grid_log_reg.best_params_)
print("Best Cross-Validation Score:", grid_log_reg.best_score_)

# Test set evaluation
y_pred_log_reg = grid_log_reg.best_estimator_.predict(X_test_scaled)
print("Test Set Performance:")
print(classification_report(y_test, y_pred_log_reg))

log_model_performance("Logistic Regression", y_test, y_pred_log_reg)

```

Best Parameters for Logistic Regression: {'C': 1, 'solver': 'liblinear'}

Best Cross-Validation Score: 0.9285714285714285

Test Set Performance:

	precision	recall	f1-score	support
0	0.88	0.86	0.87	35
1	0.86	0.89	0.87	35
accuracy			0.87	70
macro avg	0.87	0.87	0.87	70
weighted avg	0.87	0.87	0.87	70

```

/opt/anaconda3/envs/Digital_Biomarker_New/lib/python3.10/site-
packages/sklearn/linear_model/_sag.py:348: ConvergenceWarning: The max_iter was
reached which means the coef_ did not converge
warnings.warn(

```

5.3.3 Support Vector Machine (SVM)

Here, I try SVM with a hyperparameter grid and 5-fold CV. The best CV Score is 92%,

```

[33]: from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.metrics import classification_report

# Define the SVM model
svm = SVC(random_state=42)

# Define the hyperparameter grid

```

```

param_grid_svm = {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf'],
    'gamma': ['scale', 'auto']
}

# Perform grid search with 5-fold CV
grid_svm = GridSearchCV(estimator=svm, param_grid=param_grid_svm, cv=5,
    ↪scoring='accuracy')
grid_svm.fit(X_train_scaled, y_train)

# Best parameters and best score
print("Best Parameters for SVM:", grid_svm.best_params_)
print("Best Cross-Validation Score:", grid_svm.best_score_)

# Test set evaluation
y_pred_svm = grid_svm.best_estimator_.predict(X_test_scaled)
print("Test Set Performance:")
print(classification_report(y_test, y_pred_svm))
log_model_performance("SVM", y_test, y_pred_svm)

```

Best Parameters for SVM: {'C': 1, 'gamma': 'scale', 'kernel': 'linear'}

Best Cross-Validation Score: 0.9321428571428573

Test Set Performance:

	precision	recall	f1-score	support
0	0.91	0.89	0.90	35
1	0.89	0.91	0.90	35
accuracy			0.90	70
macro avg	0.90	0.90	0.90	70
weighted avg	0.90	0.90	0.90	70

5.3.4 Random Forest

Next step is trying a Random Forest Classifier with hyperparameter grid search and 5-fold CV.

```
[34]: rf = RandomForestClassifier(random_state=42)
```

```

# hyperparameter grid
param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

```

```

grid_rf = GridSearchCV(estimator=rf, param_grid=param_grid_rf, cv=5,
    ↪scoring='accuracy', n_jobs=-1)
grid_rf.fit(X_train, y_train)

print("Best Parameters for Random Forest:", grid_rf.best_params_)
print("Best Cross-Validation Score:", grid_rf.best_score_)

# Test set evaluation
y_pred_rf = grid_rf.best_estimator_.predict(X_test)
print("Test Set Performance:")
print(classification_report(y_test, y_pred_rf))
log_model_performance("Random Forest", y_test, y_pred_rf)

```

Best Parameters for Random Forest: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 300}

Best Cross-Validation Score: 0.9464285714285714

Test Set Performance:

	precision	recall	f1-score	support
0	0.86	0.89	0.87	35
1	0.88	0.86	0.87	35
accuracy			0.87	70
macro avg	0.87	0.87	0.87	70
weighted avg	0.87	0.87	0.87	70

The accuracy of the Random Forest in Crossvalidation is 92% with an (almost) equal prediction power for both classes. This could be a promising model!

5.3.5 MLP

I'm training a multilayer perceptron, a kind of NN that is composed of fully connected layers.

```

[35]: mlp = MLPClassifier(max_iter=2000, random_state=42)

# Define the hyperparameter grid
param_grid_mlp = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50), (100, 100)],
    'activation': ['tanh', 'relu'],
    'solver': ['adam', 'sgd'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive']
}

```

```

grid_mlp = GridSearchCV(estimator=mlp, param_grid=param_grid_mlp, cv=5,
↳ scoring='accuracy', n_jobs=-1)
grid_mlp.fit(X_train_scaled, y_train)

print("Best Parameters for MLP:", grid_mlp.best_params_)
print("Best Cross-Validation Score:", grid_mlp.best_score_)

y_pred_mlp = grid_mlp.best_estimator_.predict(X_test_scaled)
print("Test Set Performance:")
print(classification_report(y_test, y_pred_mlp))
log_model_performance("MLP", y_test, y_pred_mlp)

import os
os.system('say "Training is complete"') # Uses macOS text-to-speech

```

Best Parameters for MLP: {'activation': 'tanh', 'alpha': 0.0001, 'hidden_layer_sizes': (100, 100), 'learning_rate': 'constant', 'solver': 'adam'}

Best Cross-Validation Score: 0.95

Test Set Performance:

	precision	recall	f1-score	support
0	0.88	0.86	0.87	35
1	0.86	0.89	0.87	35
accuracy			0.87	70
macro avg	0.87	0.87	0.87	70
weighted avg	0.87	0.87	0.87	70

[35]: 0

5.3.6 TPOT

Lastly, I try using the TPOT (Tree-based Pipeline Optimization Tool) python library that helps automate the selection of a machine learning pipeline. It returns the best pipeline.

```

[36]: # Function to suppress printed output (tpot keeps printing unwanted messages
↳ "is_classifier is_regressor")
@contextlib.contextmanager
def suppress_stdout():
    with open(os.devnull, 'w') as devnull:
        old_stdout = sys.stdout
        sys.stdout = devnull
        try:
            yield
        finally:
            sys.stdout = old_stdout

```

```

tpot = TPOTClassifier(
    generations=5,          # Number of generations to run the genetic algorithm
    population_size=20,     # Number of pipelines to evaluate per generation
    verbosity=0,           # Verbosity level for progress updates
    scoring='accuracy',     # Evaluation metric
    random_state=42,        # For reproducibility
    n_jobs=-1              # Use all available CPU cores
)

# Run TPOT without printing unwanted messages
with suppress_stdout():
    tpot.fit(X_train_scaled, y_train)

print("Test accuracy:", tpot.score(X_test_scaled, y_test))

#tpot.export('tpot_best_pipeline.py')

```

```

/opt/anaconda3/envs/Digital_Biomarker_New/lib/python3.10/site-
packages/sklearn/base.py:1230: FutureWarning: passing a class to None is
deprecated and will be removed in 1.8. Use an instance of the class instead.
    warnings.warn(
/opt/anaconda3/envs/Digital_Biomarker_New/lib/python3.10/site-
packages/sklearn/base.py:1270: FutureWarning: passing a class to None is
deprecated and will be removed in 1.8. Use an instance of the class instead.
    warnings.warn(

```

Test accuracy: 0.8571428571428571

```

[37]: y_pred = tpot.predict(X_test)
      log_model_performance("TPOT", y_test, y_pred)

```

```

/opt/anaconda3/envs/Digital_Biomarker_New/lib/python3.10/site-
packages/sklearn/utils/validation.py:2732: UserWarning: X has feature names, but
ExtraTreesClassifier was fitted without feature names
    warnings.warn(

```

5.3.7 Comparing Model Performance

Now I execute the function I previously defined to be able to compare the different models.

```

[38]: comparison_table = display_performance_comparison()

```

Model Performance Comparison:

	Model	Accuracy	Precision	Recall	F1-Score
1	SVM	0.900000	0.900327	0.900000	0.899980
0	Logistic Regression	0.871429	0.871732	0.871429	0.871402

2	Random Forest	0.871429	0.871732	0.871429	0.871402
3	MLP	0.871429	0.871732	0.871429	0.871402
4	TPOT	0.814286	0.820833	0.814286	0.813333

6 Best Model

I chose the best model from the previous step and trained it on the training data, so I would then be able to use it for SHAP and Permutation Feature Importance.

```
[39]: rf_model = RandomForestClassifier(
        n_estimators=200,
        max_depth=None,
        min_samples_leaf=1,
        min_samples_split=2,
        random_state=42,
        n_jobs=-1 # Use all available CPU cores
    )

rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)

# Model performance
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")
```

Model Accuracy: 0.8857

6.1 Classification Report

```
[52]: class_report = classification_report(y_test, y_pred)
print("Classification Report:\n", class_report)
```

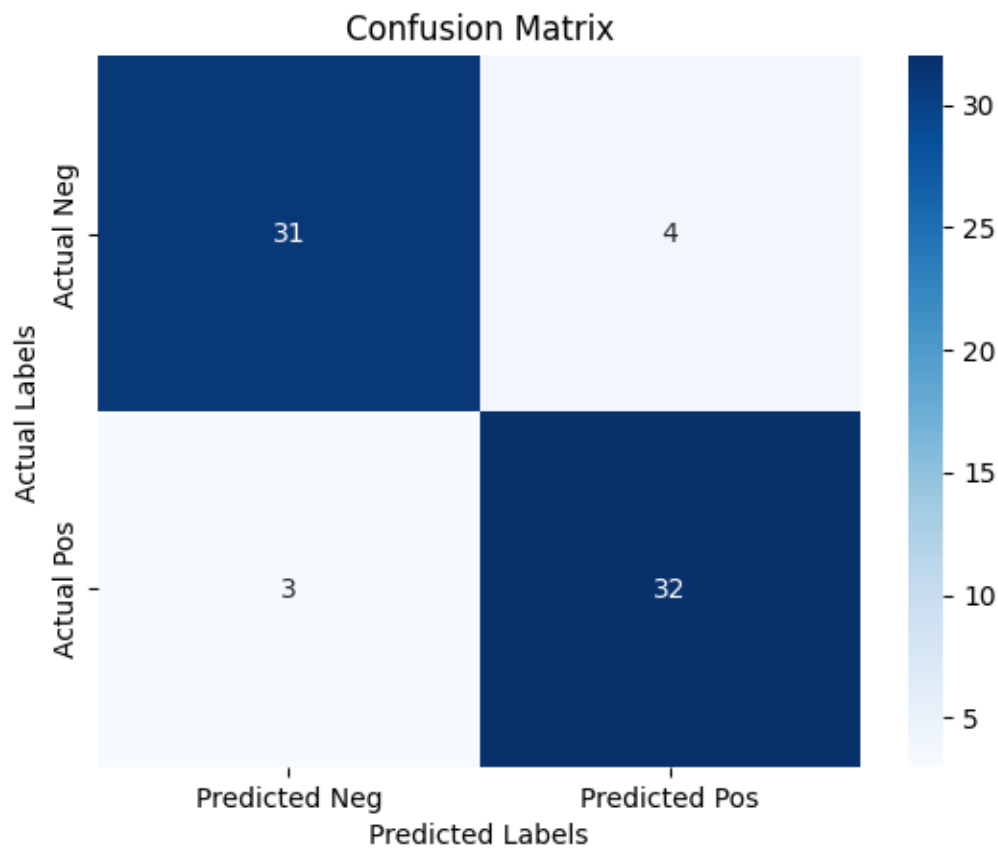
Classification Report:

	precision	recall	f1-score	support
0	0.91	0.89	0.90	35
1	0.89	0.91	0.90	35
accuracy			0.90	70
macro avg	0.90	0.90	0.90	70
weighted avg	0.90	0.90	0.90	70

6.2 Confusion Matrix

```
[53]: conf_matrix = confusion_matrix(y_test, y_pred)

sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted Neg', 'Predicted Pos'], yticklabels=['Actual Neg',
            ↪ 'Actual Pos'])
plt.xlabel('Predicted Labels')
plt.ylabel('Actual Labels')
plt.title('Confusion Matrix')
plt.show()
```



6.3 ROC_AUC

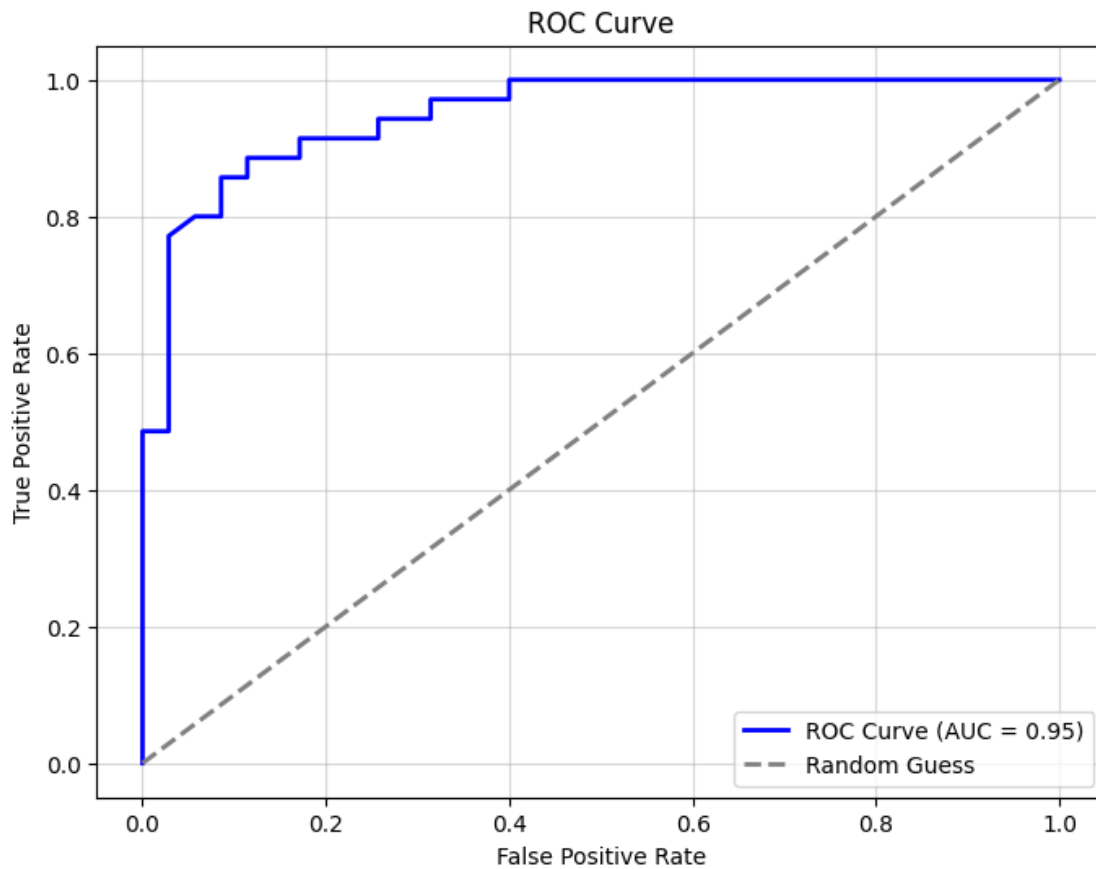
```
[54]: y_prob = rf_model.predict_proba(X_test)[: , 1] # get the probability of the
            ↪ positive class
roc_auc = roc_auc_score(y_test, y_prob)
print(f"ROC-AUC Score: {roc_auc:.2f}")
```

ROC-AUC Score: 0.95

```
[55]: fpr, tpr, thresholds = roc_curve(y_test, y_prob)

roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f"ROC Curve (AUC = {roc_auc:.2f})")
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', lw=2, label="Random
↳Guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.grid(alpha=0.5)
plt.show()
```

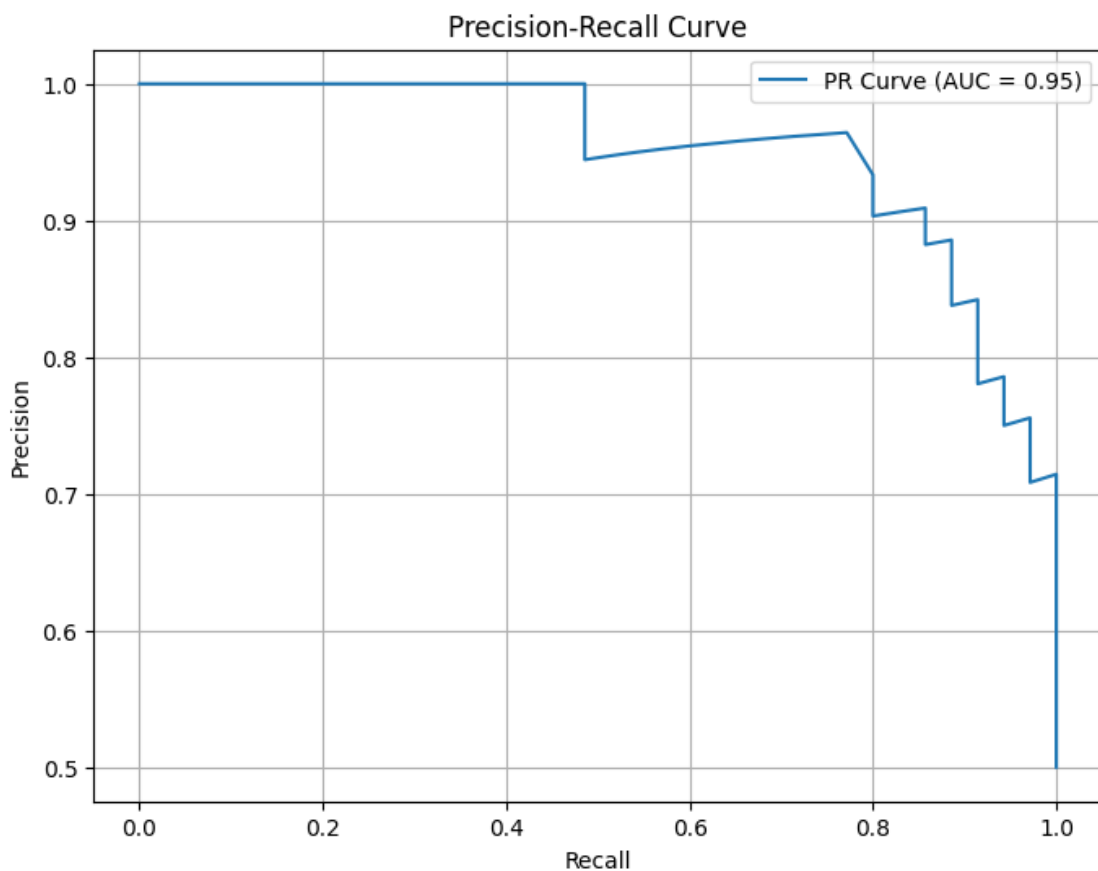


6.4 Precision-Recall Curve

Plotting the Precision against the Recall.

```
[56]: precision, recall, _ = precision_recall_curve(y_test, y_prob)
pr_auc = auc(recall, precision)

# Plot the Precision-Recall curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, label=f'PR Curve (AUC = {pr_auc:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid()
plt.show()
```



6.5 XAI Plots

I run explainable AI plots to see the influence my features have on the prediction.

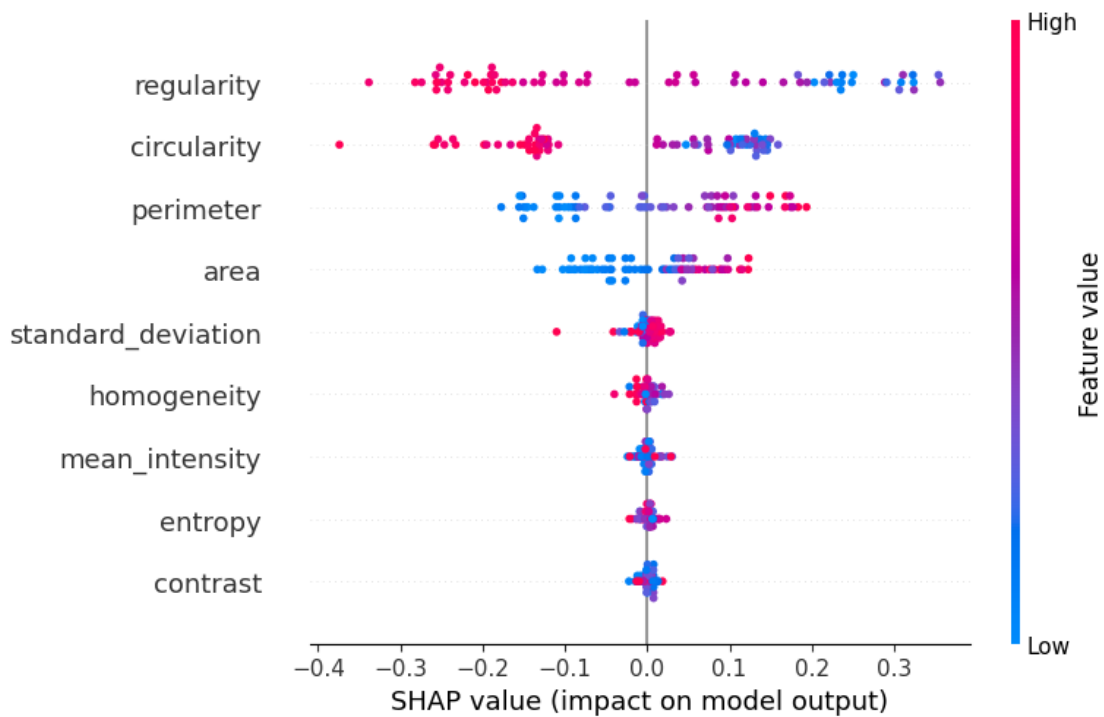
6.5.1 SHAP Summary Plot

```
[57]: import shap
import matplotlib.pyplot as plt

# SHAP analysis
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)

# Summary plot for feature importance
shap.summary_plot(shap_values[1], X_test) # For classification, selecting_
↳ class 1

# Show plot
plt.show()
```

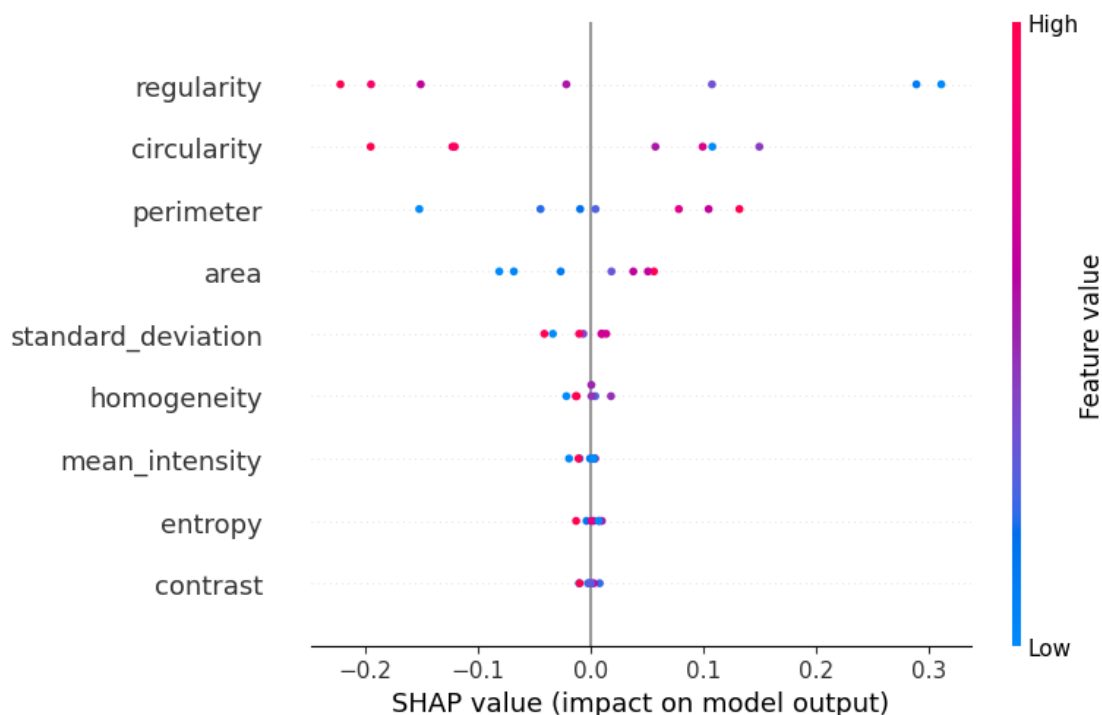


6.5.2 SHAP Summary Plot for Misclassified Instances

```
[58]: # Identify misclassified samples
misclassified_idx = np.where(y_pred != y_test)[0] # Indices of misclassified_
↳ samples
X_misclassified = X_test.iloc[misclassified_idx]
y_misclassified = y_test.iloc[misclassified_idx]
```

```
# Compute SHAP values for misclassified samples
shap_values_misclassified = explainer.shap_values(X_misclassified)

# Summary Plot for Misclassified Samples
shap.summary_plot(shap_values_misclassified[1], X_misclassified)
```



6.5.3 SHAP Waterfall Plot for Individual Misclassified Instances

```
[59]: # Identify false positives (FP) and false negatives (FN)
false_positives_idx = np.where((y_pred == 1) & (y_test == 0))[0]
false_negatives_idx = np.where((y_pred == 0) & (y_test == 1))[0]

X_fp = X_test.iloc[false_positives_idx]
X_fn = X_test.iloc[false_negatives_idx]

# Compute SHAP values for FP and FN cases
shap_values_fp = explainer.shap_values(X_fp)
shap_values_fn = explainer.shap_values(X_fn)

# Generate waterfall plots for False Positives
print("\nFalse Positives:")
for i in range(len(X_fp)):
    print(f"False Positive {i + 1}/{len(X_fp)}")
    shap.waterfall_plot(shap.Explanation(values=shap_values_fp[1][i],
```

```

expected_value[1],
base_values=explainer.

data=X_fp.iloc[i].values,
feature_names=X_fp.columns))

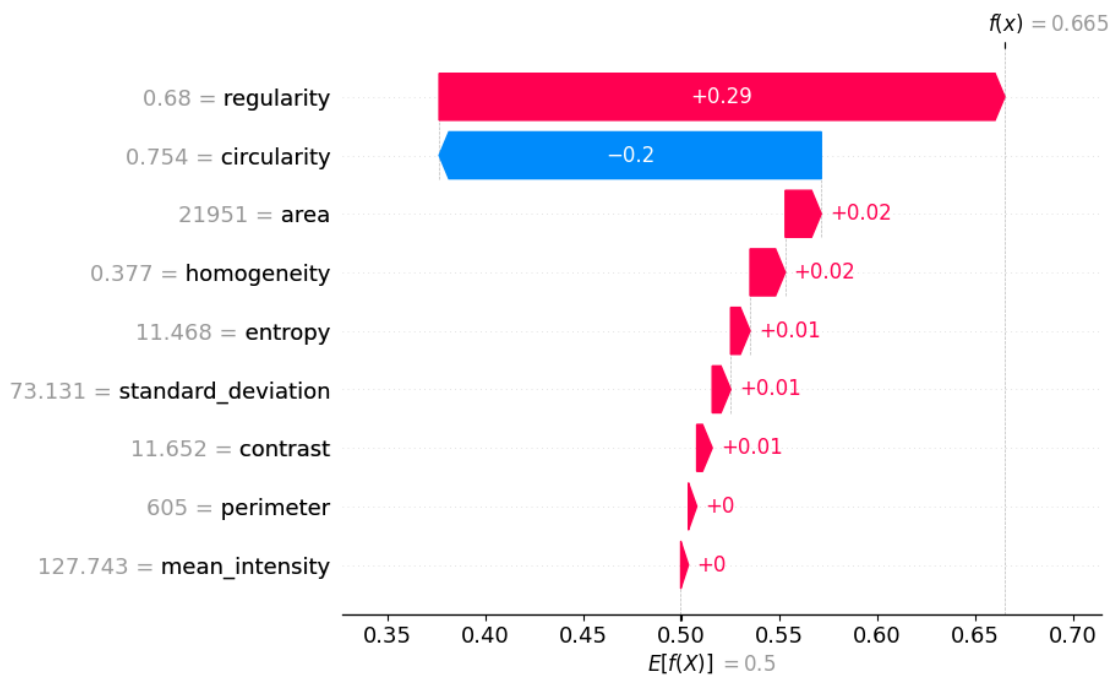
# Generate waterfall plots for False Negatives
print("\nFalse Negatives:")
for i in range(len(X_fn)):
    print(f"False Negative {i + 1}/{len(X_fn)}")
    shap.waterfall_plot(shap.Explanation(values=shap_values_fn[1][i],
    base_values=explainer.

expected_value[1],

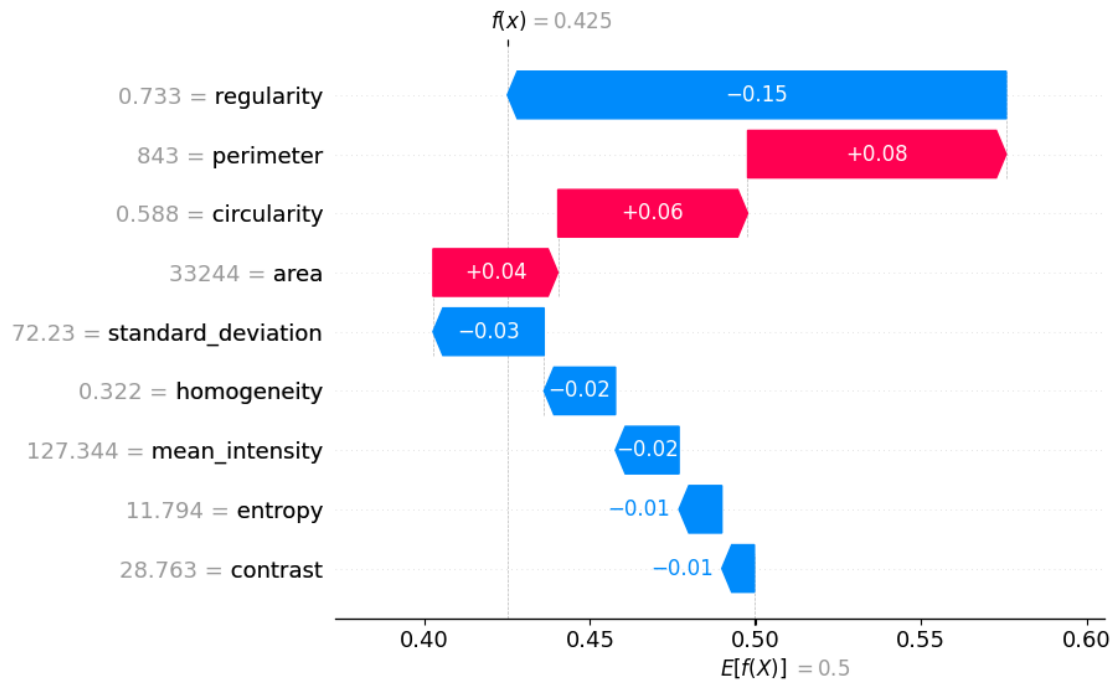
data=X_fn.iloc[i].values,
feature_names=X_fn.columns))

```

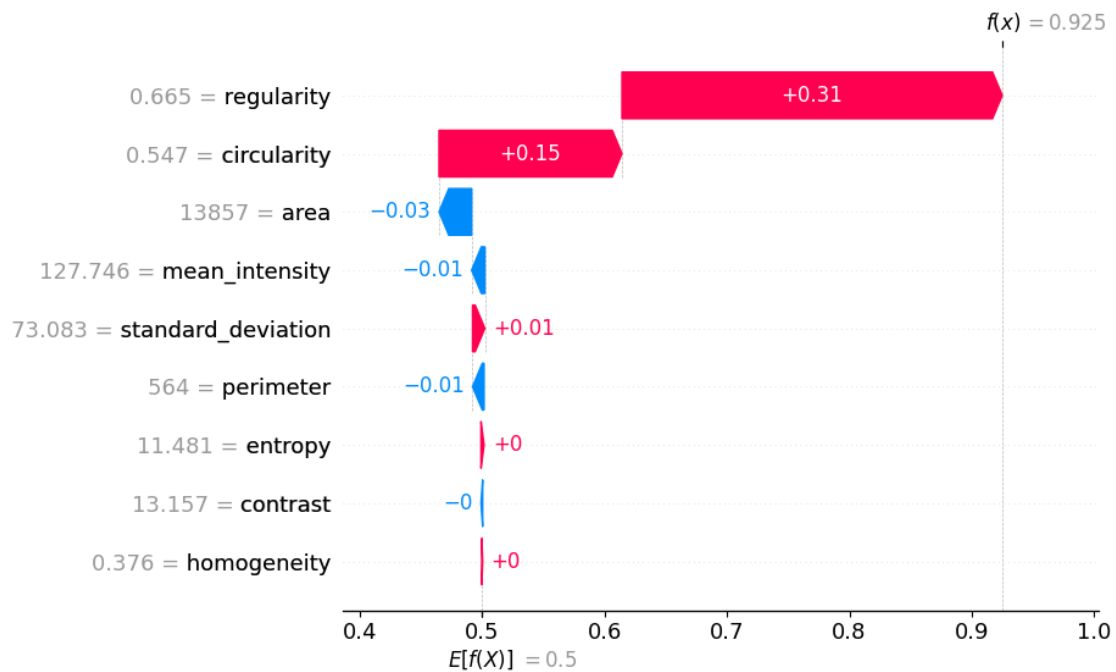
False Positives:
False Positive 1/4



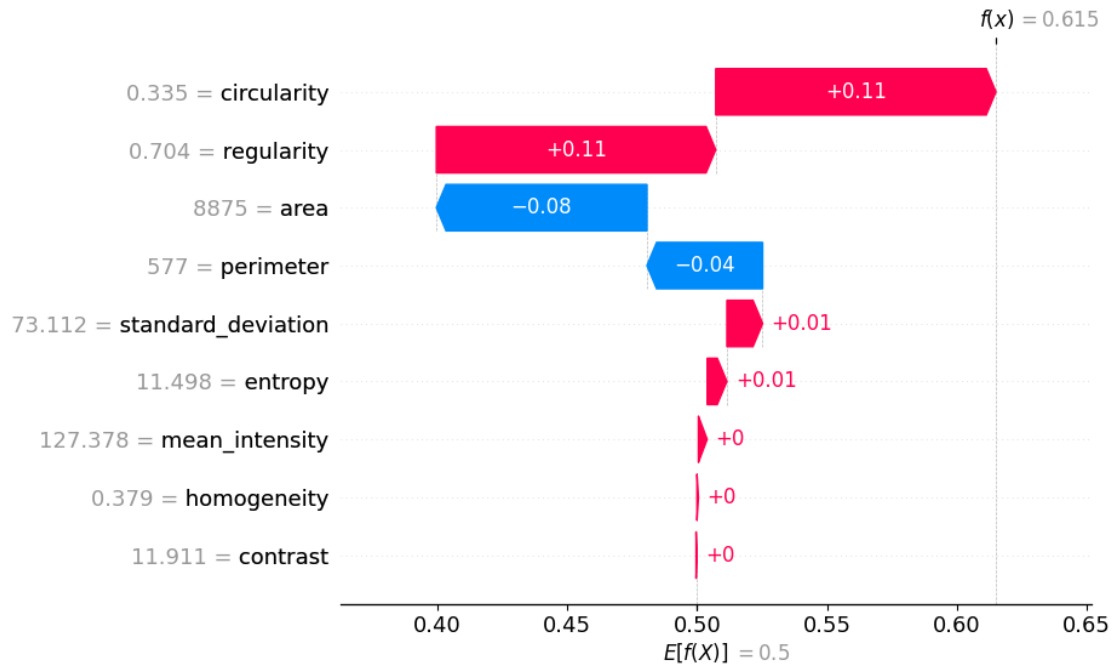
False Positive 2/4



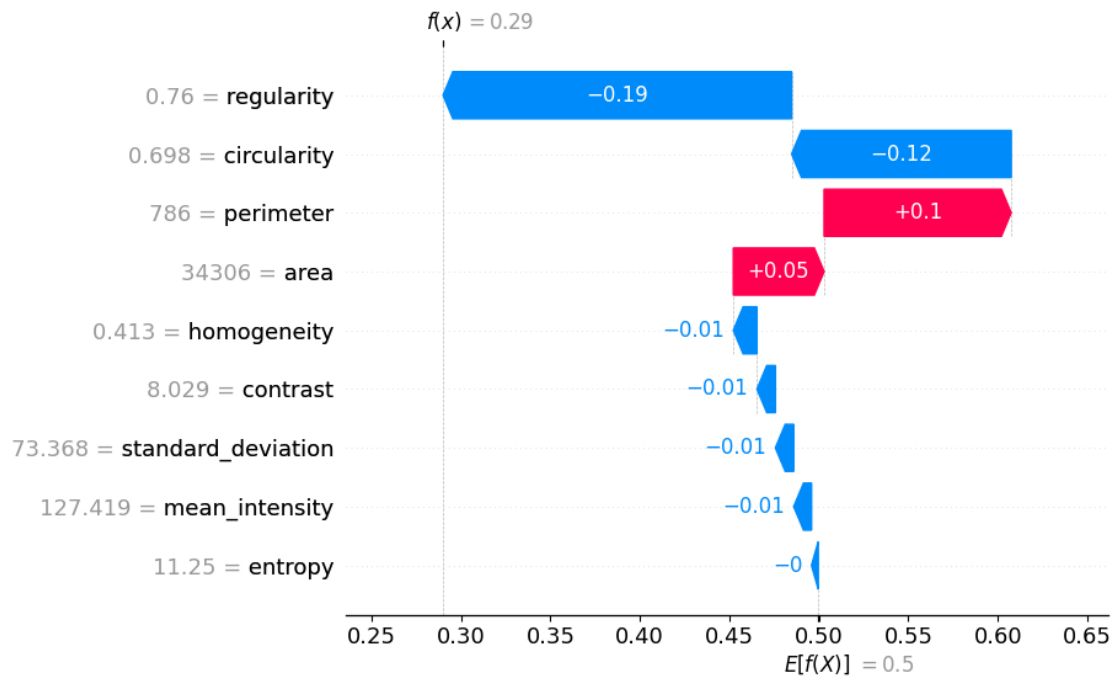
False Positive 3/4



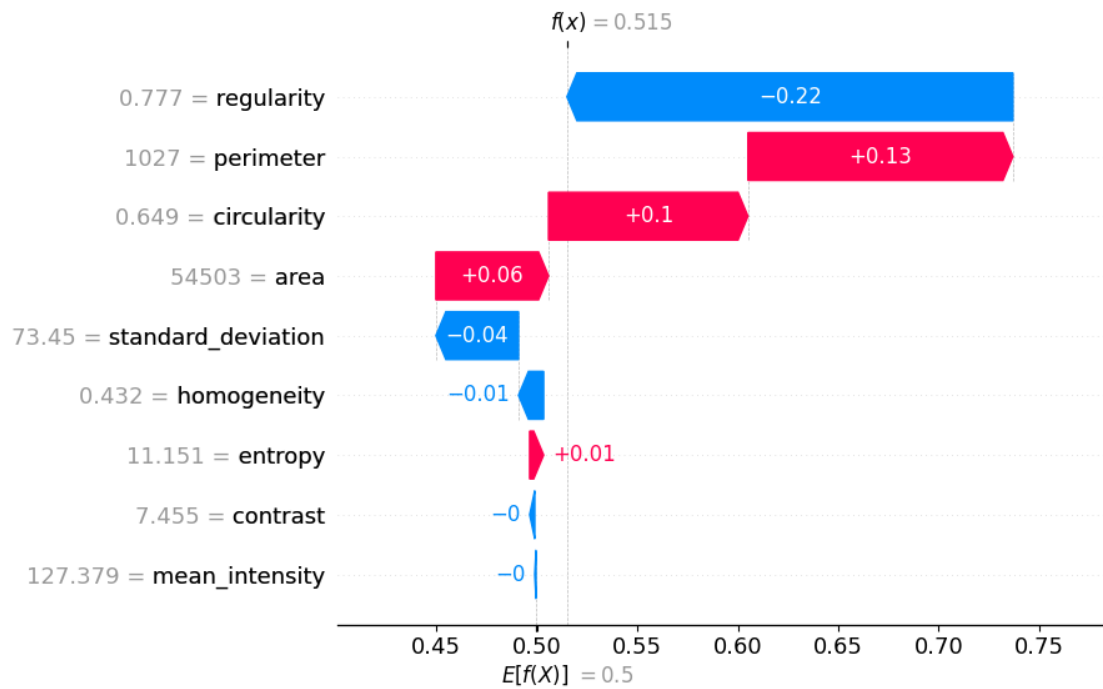
False Positive 4/4



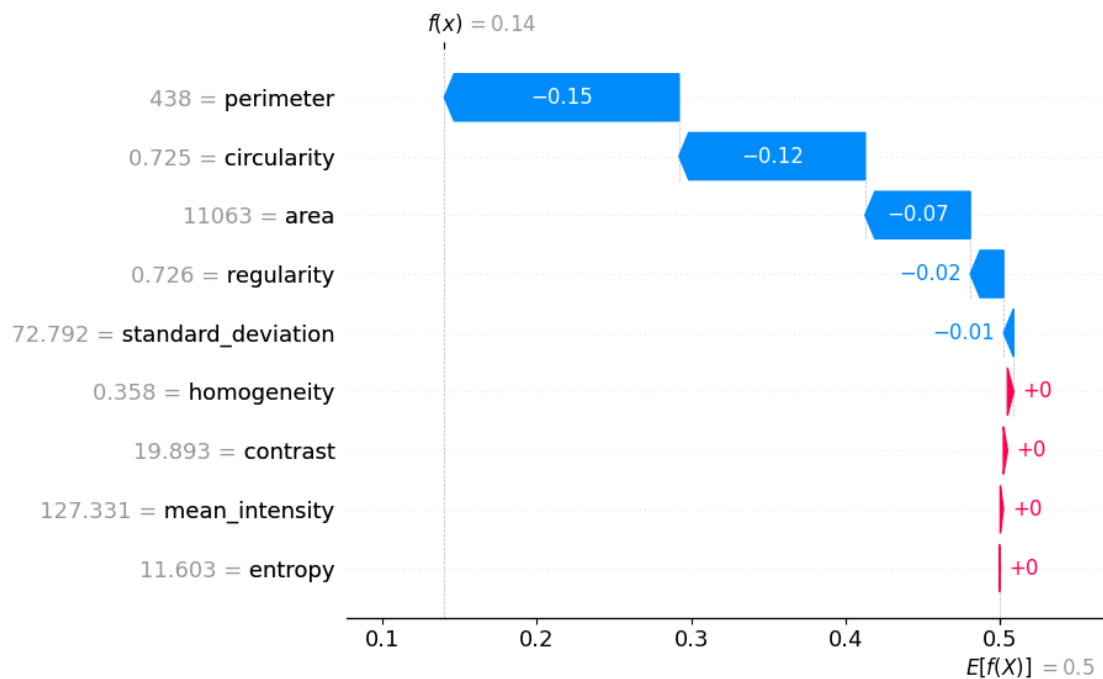
False Negatives:
False Negative 1/3



False Negative 2/3



False Negative 3/3



6.5.4 Permutation Feature Importance Plot

```
[60]: from sklearn.inspection import permutation_importance

perm_importance = permutation_importance(rf_model, X_test, y_test,
    ↪n_repeats=10, random_state=42, n_jobs=-1)

# Convert results into a DataFrame
perm_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': perm_importance.importances_mean
}).sort_values(by='Importance', ascending=False)

# Plot Permutation Feature Importance
plt.figure(figsize=(10, 6))
plt.barh(perm_df['Feature'], perm_df['Importance'], align='center')
plt.xlabel("Permutation Importance")
plt.ylabel("Feature")
plt.title("Permutation Feature Importance (Mean)")
plt.gca().invert_yaxis() # Reverse order for better visualization
plt.show()
```

